	Institut Teknologi Bandung Program Studi Teknik Fisika		
PRAKTIKUM	TF4061 Industrial Internet of Things	2+1 SKS	Dr. Ir. Eko Mursito Budi, MT IPM Faqihza Mukhlish, S.T., M.T., Ph.D

3 NODEJS DAN NDE MCU

3.1 LATAR BELAKANG

Server adalah sistem *software* dan *hardware* komputer yang didedikasikan untuk melayani: menerima *request* dan memberi *response* dari/ke *client* dalam suatu jaringan komputer. Server dibangun berdasarkan keperluan yang dijalankan, misalnya untuk *website*, *database*, transfer file, e-mail, dan aplikasi web lainnya, termasuk IOT. IOT-Server merupakan komponen penting dalam arsitektur IOT untuk melayani proses dari/ke IOT-Node dan menjalankan proses lain yang diperlukan.

Saat ini tersedia berbagai *software* untuk membangun *server*, misalnya Nginx, Apache, dan Node.js. Node.js sendiri lebih tepat disebut sebagai *runtime environment*, dimana aplikasi *server* dapat dirancang di dalamnya. IOT-Server dalam praktikum ini dibangun menggunakan Node.js.

3.2 CAPAIAN PEMBELAJARAN

a. Mahasiswa memahami bagaimana MQTT server bekerja
b. Mahasiswa dapat membangun IoT Node, publisher dan subscriber.

3.3 ALAT & BAHAN

No	Item	Banyak	Keterangan
1	Komputer / laptop	1	Disediakan peserta
2	Jaringan Internet	1	
3	ESP32	1	
4	Sofware terinstall seperti pada tugas awal	3	

3.4 PANDUAN TEKNIS

- NodeJs: https://nodejs.org/
- Visual Studio Code: https://code.visualstudio.com
- Chrome: https://www.google.co.id/chrome/
- Arduino IDE: https://wiki-content.arduino.cc/en/software
- MQTT-Explorer: http://mqtt-explorer.com

3.5 TUGAS AWAL

Pada komputer / laptop anda, install program:

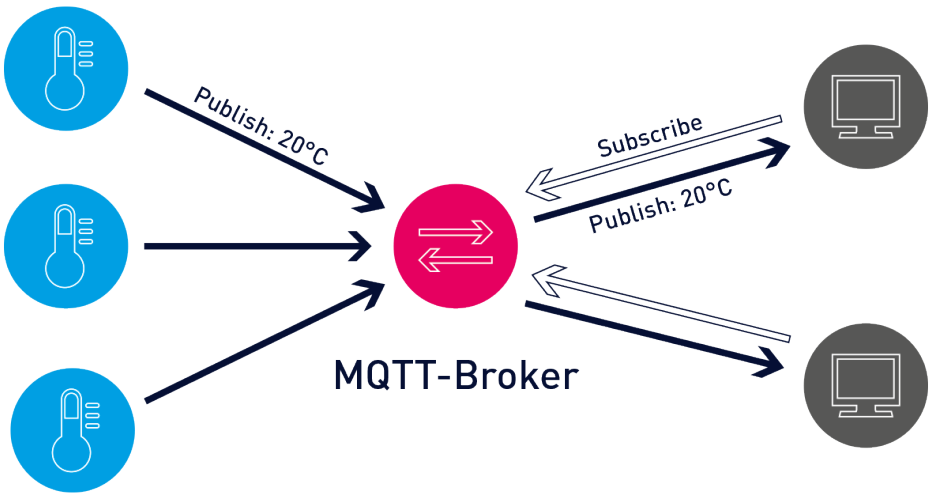
- Nodejs versi LTS terbaru
- Code editor, disarankan visual studio code
- Web browser, disarankan Chrome
- Arduino IDE
- MQTT Explorer

3.6 PRAKTIKUM

3.6.1 MQTT PROTOCOL - BROKER

Pada praktikum sebelumnya, pembahasan tentang HTTP request respond telah dilakukan. Selanjutnya, pada praktikum kita kali ini, praktikan akan mempelajari bagaimana protokol komunikasi MQTT bekerja dengan menggunakan lingkungan pengembangan NodeJS. Seperti yang sudah dibahas pada materi kuliah, MQTT memiliki 3 entitas utama, seperti **Gambar 4-1**, yaitu:

a. MQTT Publisher: Sebuah fungsi yang dapat digunakan oleh IoT Node untuk mengirimkan "message" atau data berdasarkan topic.
b. MQTT Subscriber: Sebuah fungsi yang dapat digunakan oleh IoT Node untuk menerima "message" atau data berdasarkan topic.
c. MQTTBroker: Sebuah server yang berfungsi untuk mengatur alur "message" atau data berdasarkan topic diantara publisher dan subscriber.



Gambar 3-1. Alur "message" atau data pada protokol MQTT

Supaya alur data dapat dilakukan dengan menggunakan protokol MQTT, entitas yang perlu dibangun diawal adalah broker. Siapkan project NodeJS dan library aedes.

1) Buatlah project NodeJS dan install modul aedes melalui npm

```
$ npm install aedes
```

2) Selanjutnya, buatlah file javascript baru dengan nama broker.js.

3) Lalu, buatlah program berikut yang merupakan MQTT server sederhana

Kode 3-1: broker.js

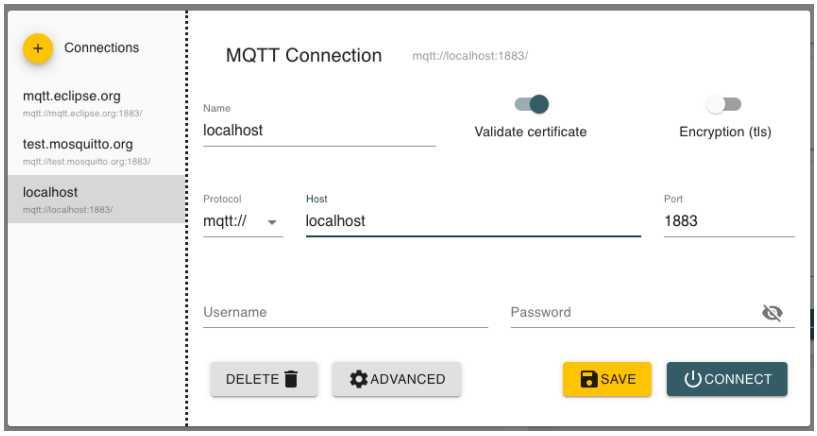
```
const aedes = require('aedes')()
const broker = require('net').createServer(aedes.handle)
const broker_port = 1883

broker.listen(broker_port, function () {
  console.log('MQTT broker started and listening on port ', broker_port)
})
```

4) Jalankan broker.js dengan menggunakan command line node broker.js lalu tekan **enter** seperti contoh berikut pada terminal. Jika berhasil, output pada terminal akan muncul seperti baris kedua pada command line di bawah ini.

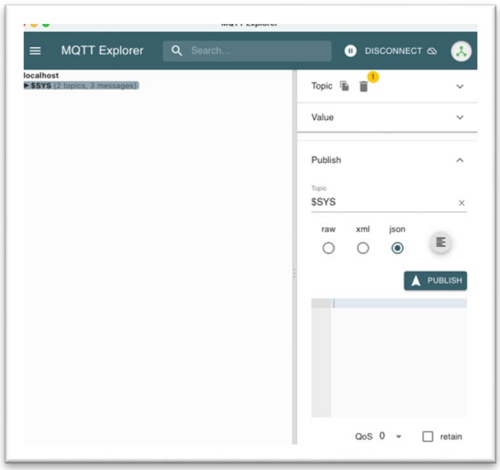
```
$ node broker.js
MQTT broker started and listening on port  1883
```

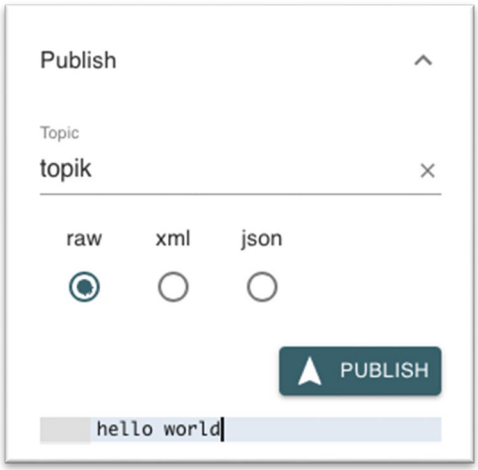
5) Selanjutnya, silahkan jalankan aplikasi MQTT-explorer dan isi pengaturan seperti pada berikut:



Gambar 3-2. Pengaturan MQTT Connection pada MQTT Explorer

6) Save dan Connect, sehingga tampilan menjadi seperti berikut:

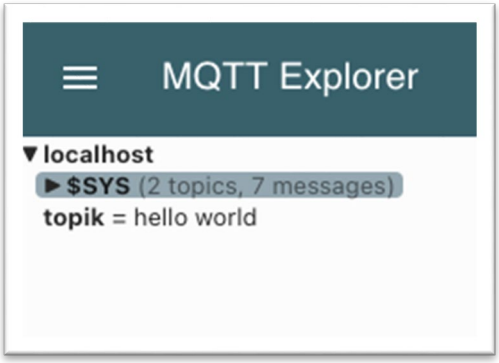




Gambar 3-3. Tampilan utama MQTT explorer

Gambar 3-4. Publish data dan topik

7) Jika program broker.js berhasil, tampilan pada kolom monitor akan muncul topik dan data pada MQTT server



Gambar 3-5. Data dan topik pada monitor

TUGAS - 1:
Lakukan pengiriman dari komputer/laptop teman anda dengan menggunakan MQTT Explorer. Berikut adalah kisi-kisinya:
1. Matikan firewall pada PC yang menjalankan broker.
2. Catat IP PC yang menjalan broker dengan ipconfig pada cmd (windows) atau ifconfig pada bash (unix)
3. Koneksikan komputer/laptop lain pada jaringan yang sama dengan broker.
4. Pada komputer kedua koneksikan MQTT explorer dengan IP PC broker dan port 1883
5. Kirimkan topik dan data dari komputer kedua.
6. Amati perilakunya.

3.6.2 MQTT PUBLISHER

NodeJS dapat berjalan dengan beberapa eksekusi secara parallel. Pada praktikum ini, biarkan broker.js yang sudah dibangun pada praktikum sebelumnya. Lalu, praktikan akan membangun aplikasi NodeJS yang berfungsi sebagai publisher.

1) Untuk membangun publisher, library yang digunakan adalah **mqtt** dengan cara

```
$ npm install mqtt
```

2) Buatlah file baru publisher.js seperti berikut:

Kode 3-2: publisher.js

```
const mqtt = require('mqtt');
const port = 8000;
const broker_address = 'mqtt://localhost:1883'

const client = mqtt.connect(broker_address);

var data = 0;
```

```
client.on('connect',function(){
            console.log("Publishing to $s",broker_address);

            setInterval(function(){
                        data++;
                        let message = `${data}`;
           let topic = "topik";
           client.publish(topic, message);
    },100)
});
```

3) Jalankan aplikasi publisher.js dengan cara berikut:

```
$ node publisher.js
```

4) Jalankan MQTT explorer dan hubungkan ke broker, perhatikan apa yang terjadi.

TUGAS - 2:
Lakukan pengiriman dengan publisher.js dari komputer/laptop teman anda.
1. Gunakan pengaturan komputer broker seperti pada praktikum sebelumnya.
2. Gubah broker_address pada publisher.js dengan IP broker.

3.6.3 MQTT SUBSCRIBER VIA WEBSOCKET

Pada Praktikum ini akan dibangun tiga entitas, yaitu broker, publisher dan subscriber.

a. Broker terdiri dari broker.js yang mempunyai konektivitas MQTT via WebSocket.
b. Publisher berupa publisher.js dengan menggunakan **Math.Sin();**
c. Subscriber berupa aplikasi index.js dengan respond berupa index.html dan script.js. Modul paho digunakan untuk mengaktifkan MQTT client via websocket.

Berikut ini adalah listing kode untuk setiap komponen.

Broker

Kode 3-3: broker.js

```
const aedes = require('aedes')()
const broker = require('net').createServer(aedes.handle)
const broker_port = 1883
const websocket_port = 8883

const httpServer = require('http').createServer()
```

```
const ws = require('websocket-stream')
ws.createServer({ server: httpServer }, aedes.handle)

httpServer.listen(websocket_port, function () {
  console.log('Aedes MQTT-WS listening on port: ' + websocket_port)
});

broker.listen(broker_port, function () {
  console.log('MQTT broker started and listening on port ', broker_port)
})
```

Publisher

Kode 3-4: publisher.js

```
const http = require("http");
const mqtt = require('mqtt');
const fs = require("fs");
const host = 'localhost';
const port = 8000;

const broker_address = 'mqtt://localhost:1883'

const client = mqtt.connect(broker_address);

var t = 0;

client.on('connect',function(){
	console.log("Publishing to $s",broker_address);

	setInterval(function(){
		t++;
		let y = Math.round(50+20*Math.sin(0.2*t));
		let message = y.toString();
		let topic = "topik";
	client.publish(topic, message);
	},100)
});
```

Subscriber

Kode 3-5: index.html

```
<!doctype html>
<html>
    <head>
        <style>
            #container {
                width:70px;
                height:300px;
                line-height:300px;
                margin-left:10px;
                border:2px solid black;
                background-color: white;
            }

            #top {
                width: 100%;
                background-color:white;
                text-align:center;
            }
        </style>

        <script type="text/javascript" src="https://cdnjs.cloudflare.com/ajax/libs/paho-
mqtt/1.0.1/mqttws31.min.js"></script>
        <script type="text/javascript" src="script.js" charset="utf-8"></script>

        <meta charset="utf-8">
        <title>Demo TF4061 IIOT - Week 2</title>
    </head>

    <body>
        <h3>Publish any value between 0-100 at MQTT topic "/test"</h3>
        <div id="container">
            <div id="top">(No data)</div>
        </div>
    </body>
</html>
```

Kode 3-6: script.js

```
var hostname = "localhost";
var port = 8883;
var clientId = "WebSocket";
clientId += new Date().getUTCMilliseconds();;
var topic = "topik";
```

```
mqttClient = new Paho.MQTT.Client(hostname, port, clientId);
mqttClient.onMessageArrived = MessageArrived;
mqttClient.onConnectionLost = ConnectionLost;
Connect();

/*Initiates a connection to the MQTT broker*/
function Connect(){
    mqttClient.connect({
    onSuccess: Connected,
    onFailure: ConnectionFailed,
    keepAliveInterval: 10,
});
}

/*Callback for successful MQTT connection */
function Connected() {
    console.log("Connected to MQTT-over-WebSocket broker.");
    mqttClient.subscribe(topic);
}

/*Callback for failed connection*/
function ConnectionFailed(res) {
            console.log("Connect failed:" + res.errorMessage);
}

/*Callback for lost connection*/
function ConnectionLost(res) {
    if (res.errorCode !== 0) {
        console.log("Connection lost:" + res.errorMessage);
        Connect();
    }
}

/*Callback for incoming message processing */
function MessageArrived(message) {
    console.log(message.destinationName +" : " + message.payloadString);

    var a = parseInt(message.payloadString);
    var ht = 100-a;
    document.getElementById("top").style.height = ""+ht+"%" ;
    document.getElementById("top").innerHTML = message.payloadString+"%";
    document.getElementById("container").style.backgroundColor = "#74add6";
}
```

Kode 3-7: index.js

```
const http = require("http");
const fs = require("fs");
var url = require("url");
const host = 'localhost';
const port = 8080;

const requestListener = function(request,response){
	var pathname = url.parse(request.url).pathname;
	console.log("Request for " + pathname + " received.");

	response.writeHead(200);

	if(pathname == "/") {
		html = fs.readFileSync("index.html", "utf8");
		response.write(html);
	} else if (pathname == "/script.js") {
		script = fs.readFileSync("script.js", "utf8");
		response.write(script);
	}


	response.end();
}

const server = http.createServer(requestListener);

server.listen(port,host,() =>{
	console.log(`Server is running on http://${host}:${port}`);
});
```

TANTANGAN - 1:
Buatlah aplikasi chat dengan menggunakan MQTT antara dua buah browser pada komputer berbeda dan satu komputer sebagai broker.

3.6.4 IOT NODE: ESP32 KONEKSI WIFI

Untuk melakukan koneksi ke suatu WiFi access point, kita dapat menggunakan Pustaka WiFi yang sudah tersedia pada Arduino ESP32.

```
include <WiFi.h>
```

Untuk menggunakan ESP32 sebagai WiFi station atau WiFi client yang akan terkoneksi ke jaringan, kita menggunakan mode WIFI_STA:

```
WiFi.mode(WIFI_STA);
```

ESP32 juga dapat bertindak sebagai access point (dengan mode WIFI_AP) di mana device lain dapat terhubung dengannya sebagai client.

Untuk memulai koneksi dengan access point, kita dapat menggunakan

```
Wifi.begin(ssid, password);
```

Di mana ssid dan password adalah string atau char* yang berisi nama dan password dan access point. Untuk mengecek apakah koneksi sudah berhasil atau tidak kita dapat menggunakan fungsi WiFi.status(). Jika nilai keluaran fungsi ini adalah WL_CONNECTED maka ESP32 telah berhasil terkoneksi pada access point dan mendapatkan alamat IP. Alamat IP dapat diakses dengan menggunakan fungsi WiFi.localIP().

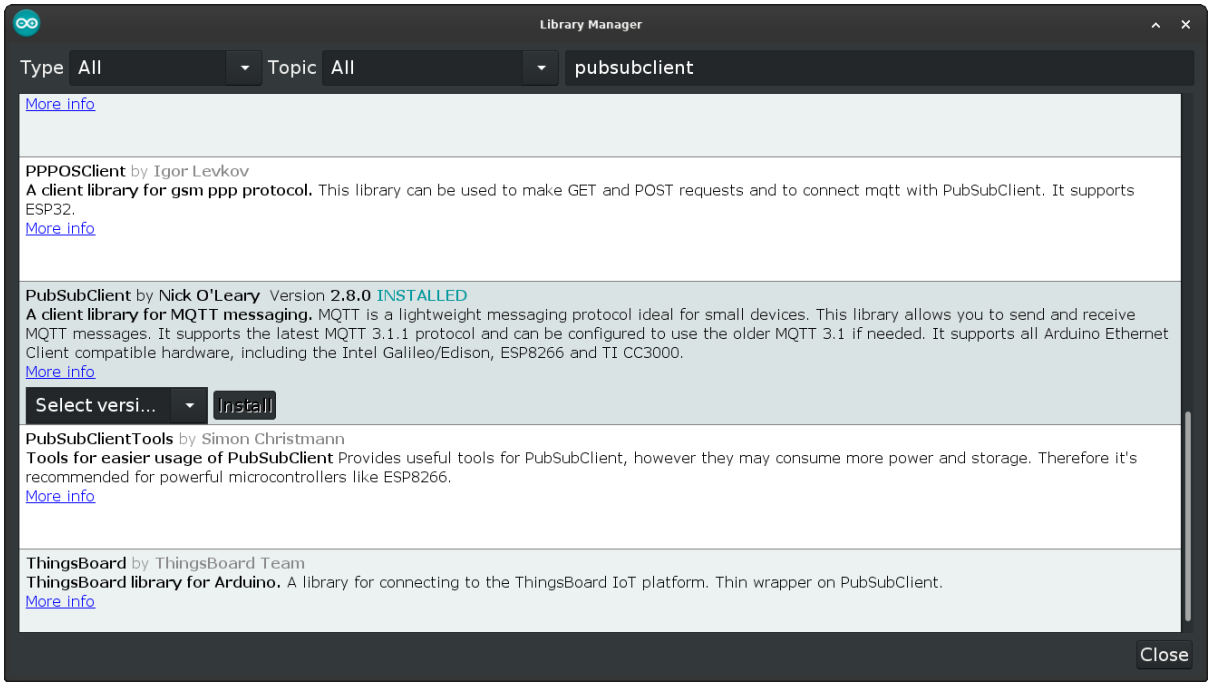
Potongan kode berikut ini akan kita gunakan.

Kode 3-8: projek.ino

```
#include <WiFi.h>
const char* ssid = "XXX"; // nama access point
const char* password = "XXX"; // passwd
void setup() {
	WiFi.mode(WIFI_STA);
	WiFi.begin(ssid, password);
	while( WiFi.status() != WL_CONNECTED ) {
		delay(500);
		Serial.print(".");
	}
	Serial.println("");
	Serial.print("WiFi connected to: ");
	Serial.println(ssid);
	Serial.println("IP address: ");
	Serial.println(WiFi.localIP());
}
void loop() {
}
```

3.6.5 IOT NODE: ESP32 SEBAGAI MQTT CLIENT

Untuk menggunakan ESP32 sebagai client dari MQTT kita perlu memasang Pustaka PubSubClient terlebih dahulu



Gambar 3-6. Pustaka pubsubclient

Untuk menggunakan Pustaka ini, kita perlu menambahkan file header pustaka yang bersangkutan:

```
#include <PubSubClient.h>
```

Kita juga perlu memastikan bahwa kita telah terhubung pada jaringan WiFi (PubSubClient memerlukan client WiFi). Misalkan kita mendeklarasikan variabel-variabel berikut untuk client WiFi dan client MQTT.

```
WiFiClient client;
PubSubClient mqttClient(client);
```

Untuk membuat koneksi pada broker atau server MQTT kita perlu mengetahui alamat dan port dari broker MQTT yang akan digunakan. Misalkan kita memiliki broker pada jaringan lokal sebagai berikut.

```
char *mqttserver = "xx.xx.xx.xx";
int mqttport = 1883;
```

Alamat server dan port tersebut akan diset pada MQTT client:

```
mqttClient.setServer(mqttServer, mqttPort);
```

Kita juga perlu mendaftarkan suatu fungsi callback yang akan dipanggil Ketika ESP32 mendapatkan pesan dari broker untuk suatu topik tertentu.

```
mqttClient.setCallback(callback);
```

Di mana callback ini adalah nama fungsi callback yang akan dipanggil. Fungsi callback ini dideklarasikan sebagai berikut

```
void callback(char* topic, byte* payload, unsigned int length){
    \\... Lakukan sesuatu sesuai topic
}
```

Sekarang kita siap untuk melakukan operasi subscribe dan publish pada ESP32. Misalkan kita mendefinisikan variabel berikut untuk nama client dan topic untuk subscriber dan publish.

```
const char* mqtt_client_name = “ESP32_nama_anda”;
const char* mqtt_pub_topic = “nama_anda_topik”;
const char* mqtt_sub_topic = “nama_topik”;
```

Pada potongan kode berikut, kita melakukan koneksi ke broker dan subscribe ke suatu topik (anda juga dapat subscribe ke beberapa topik).

```
if( !mqttClient.connected() ){
        while( !mqttClient.connected() ){
                if( mqttClient.connect(mqtt_client_name) ){
                        Serial.println("MQTT Connected!");
                        mqttClient.subscribe(mqtt_sub_topic);
                }
                else{
                        Serial.print(".");
                }
        }
}
```

Kode ini dapat diletakkan di dalam fungsi loop pada kode program anda.

Untuk mempublish data ke broker, anda dapat menggunakan perintah berikut.

```
mqttClient.publish(mqtt_pub_topic, “7);
```

Pada kode ini juga dapat diletakkan pada fungsi loop pada program anda.

Kode 3-9: projek.ino

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "XXX";
const char* password = "XXX";

// Local MQTT broker
char *mqttserver = “xxx.xxx.xxx.xxx”;
int mqttPort = 1883;

const char* mqtt_client_name = “ESP32_nama_anda”;
```

```
const char* mqtt_pub_topic = “nama_anda_topik”;
const char* mqtt_sub_topic = “nama_topik”;

WiFiClient client;
PubSubClient mqttClient(client);

void callback(char* topic, byte* payload, unsigned int length) {
	Serial.print("Message received from: ");
	Serial.println(topic);
	for (int i = 0; i < length; i++) {
		Serial.print( (char)payload[i] );
	}
	Serial.println();
	Serial.println();
}

void setup() {
	Serial.begin(115200);
	WiFi.mode(WIFI_STA);
	WiFi.begin(ssid, password);
	while( WiFi.status() != WL_CONNECTED ) {
		delay(500);
		Serial.print(".");
	}
	Serial.println("");
	Serial.print("WiFi connected to: ");
	Serial.println(ssid);
	Serial.println("IP address: ");
	Serial.println(WiFi.localIP());
	delay(2000);
	mqttClient.setServer(mqttServer, mqttPort);
	mqttClient.setCallback(callback);
}

void loop() {
	if( !mqttClient.connected() ){
		while( !mqttClient.connected() ){
			if( mqttClient.connect(mqtt_client_name) ){
				Serial.println("MQTT Connected!");
				mqttClient.subscribe(mqtt_sub_topic);
			}
			else{
				Serial.print(".");
			}
```

```
            }
        }
        mqttClient.publish(mqtt_pub_topic, "Hello I am from ESP32");
        Serial.println("Message published");
        mqttClient.loop();
        delay(5000);
}
```

TANTANGAN - 2:
• Buatlah program counter pada ESP32 yang akan bertambah jika pushbutton ditekan. Lalu, data counter ini dikirimkan ke broker dan aplikasi index.js pada praktikum sebelumnya digunakan untuk menampilkan data counter. Sehingga, anda dapat mengirimkan data counter dari ESP32 ke browser melalui MQTT. • Buatlah skenario anda dapat menyalakan dan mematikan led pada ESP32 dengan menggunakan interaksi pada browser.

3.7 TUGAS & LAPORAN

Lakukan praktikum dengan baik dan laporkan tugas dan tantangan dengan rapi!

3.8 REFERENSI

1. Repo github : https://github.com/Teknik-Fisika-ITB/IIoT_2023/tree/main/Modul_3_WebServer_NodeJS
2. Terminal tutorial: https://ubuntu.com/tutorials/command-line-for-beginners#3-opening-a-terminal
3. Cmd tutorial: https://www.freecodecamp.org/news/command-line-commands-cli-tutorial/
4. Buku: https://jonathanleemartin.com/books/